HMM Viterbi POS Tagger

Implementation, Analysis and Evaluation Report

Generated: March 10, 2026 at 17:53

1. Executive Summary

This report documents the design, implementation, and evaluation of a Hidden Markov Model (HMM) based Part-of-Speech (POS) tagger that uses the Viterbi algorithm for decoding. The system is trained on a labelled English corpus using maximum-likelihood estimation with Laplace smoothing and is evaluated on a held-out test set. Token-level accuracy and per-tag precision, recall, and F1 scores are reported together with visual analyses of the learned model parameters.

Overall test-set accuracy: 90.46% (256 / 283 tokens correctly tagged).

2. Project Workflow

The pipeline consists of four stages, each implemented as a separate Python module inside the `src/` directory:

Stage	Module	Description
1 – Data Loading & Training	src/train.py	Parse the word/TAG corpus, count transitions and emissions, estimate
2 – Decoding	src/viterbi.py	Run the Viterbi algorithm on each test sentence to find the most-probab
3 – Evaluation	src/evaluate.py	Compute token accuracy, per-tag precision/recall/F1, and build a confus
4 – Visualisation & Report	src/visualize.py report/generate_report.py	Generate analysis figures (heat-maps, bar charts) and compile this PDF

3. Dataset

The corpus uses Penn Treebank-style POS tags stored in plain-text files (one sentence per line, `word/TAG` format). A dedicated training set and a separate held-out test set are used to prevent data leakage.

3.1 Tag Set

The following Penn Treebank tags appear in the corpus:

Tag	Description
.	Sentence-final punctuation
CC	Coordinating conjunction (and, but, or)
CD	Cardinal number (one, two, three)
DT	Determiner (the, a, an, every)
IN	Preposition / subordinating conjunction (in, of, for)
JJ	Adjective (big, new, good)
JJR	Adjective, comparative (bigger, better)
NN	Noun, singular (dog, car, city)
NNP	Proper noun, singular (John, Paris, Sunday)
NNS	Noun, plural (dogs, cars, cities)
PRP	Personal pronoun (he, she, they, I)
PRP$	Possessive pronoun (his, her, their, its)
RB	Adverb (quickly, very, slowly)
RP	Particle (up, out, off)
TO	Infinitival to
VB	Verb, base form (run, eat, go)
VBD	Verb, past tense (ran, ate, went)
VBG	Verb, gerund (running, eating)
VBN	Verb, past participle (run, eaten)
VBP	Verb, present non-3rd-person (run, eat)
VBZ	Verb, present 3rd-person singular (runs, eats)

4. Model Description

4.1 Hidden Markov Model

A first-order HMM defines a joint probability over observation sequences $w_{1:n}$ and hidden state sequences $t_{1:n}$:

$$\mathbf{P(w_{1:n}, t_{1:n}) = P(t_1|START) \times \prod P(t_i|t_{i-1}) \times \prod P(w_i|t_i)}$$

The three parameter tables are:

Parameter	Symbol	Estimation
Initial probability	$\pi(t) = P(t \mid START)$	Count(START→t) / Count(START→*) [+ Laplace smoothing]
Transition probability	$A(t_i, t_j) = P(t_j \mid t_i)$	Count(t_i→t_j) / Count(t_i→*) [+ Laplace smoothing]
Emission probability	$B(t, w) = P(w \mid t)$	Count(t, w) / Count(t) [+ Laplace smoothing]

4.2 Laplace Smoothing

All probability estimates use additive (Laplace) smoothing with constant $\alpha = 1.0$. Smoothing prevents zero-probability assignments for unseen word–tag or tag–tag pairs, and handles out-of-vocabulary (OOV) words via a special <UNK> token.

4.3 Viterbi Decoding

Given a trained HMM, the Viterbi algorithm finds the most-probable tag sequence for an input sentence in $O(n \cdot |T|^2)$ time using dynamic programming. All computations are performed in log-space to prevent floating-point underflow for long sentences.

Recurrence:

$$\delta_t(j) = \max_i [\, \delta_{t-1}(i) + \log A(i,j) \,] + \log B(j, w_t)$$

The algorithm runs three passes: (1) initialisation, (2) recursion, and (3) back-trace via stored backpointers to recover the optimal path.

5. Evaluation Results

5.1 Overall Accuracy

Metric	Value
Total tokens	283
Correctly tagged	256
Token accuracy	**90.46%**

5.2 Per-Tag Metrics

Tag	Precision	Recall	F1
.	0.9259	1.0000	0.9615
DT	0.8871	1.0000	0.9402
IN	0.8889	0.8889	0.8889
JJ	1.0000	0.5000	0.6667
NN	0.8769	0.9828	0.9268
NNP	0.0000	0.0000	0.0000
NNS	0.0000	0.0000	0.0000
PRP	1.0000	1.0000	1.0000
PRP$	1.0000	1.0000	1.0000
RB	0.6667	1.0000	0.8000
TO	1.0000	0.5000	0.6667
VB	1.0000	1.0000	1.0000
VBD	0.9348	0.9149	0.9247
VBG	0.0000	0.0000	0.0000
VBZ	0.0000	0.0000	0.0000
Macro avg.	**0.6787**	**0.6524**	**0.6517**

6. Analysis and Visualisation

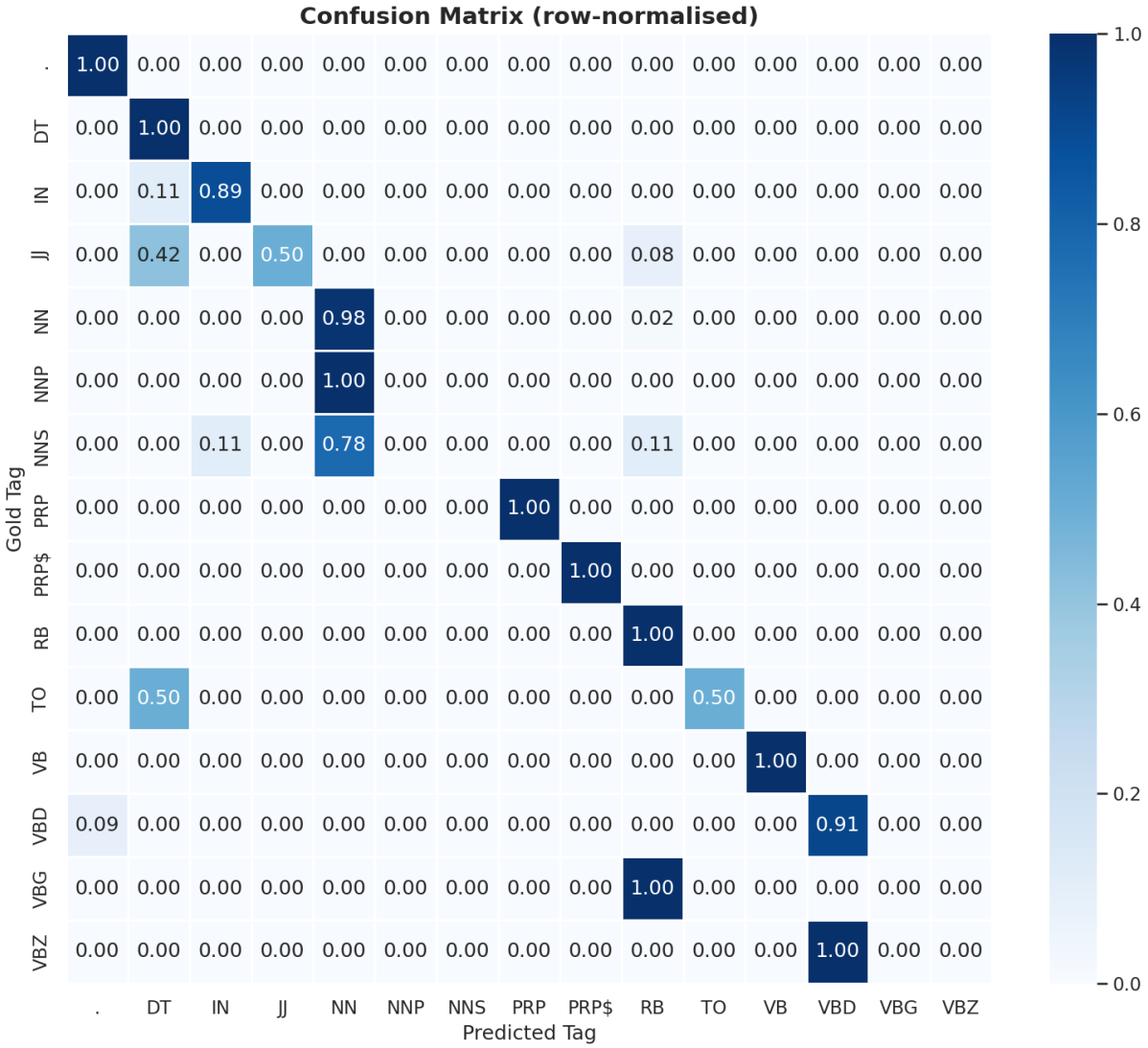

Figure 1: Confusion Matrix (row-normalised). Each cell shows the fraction of gold-tag tokens (row) predicted as each tag (column). Diagonal values represent per-tag recall.

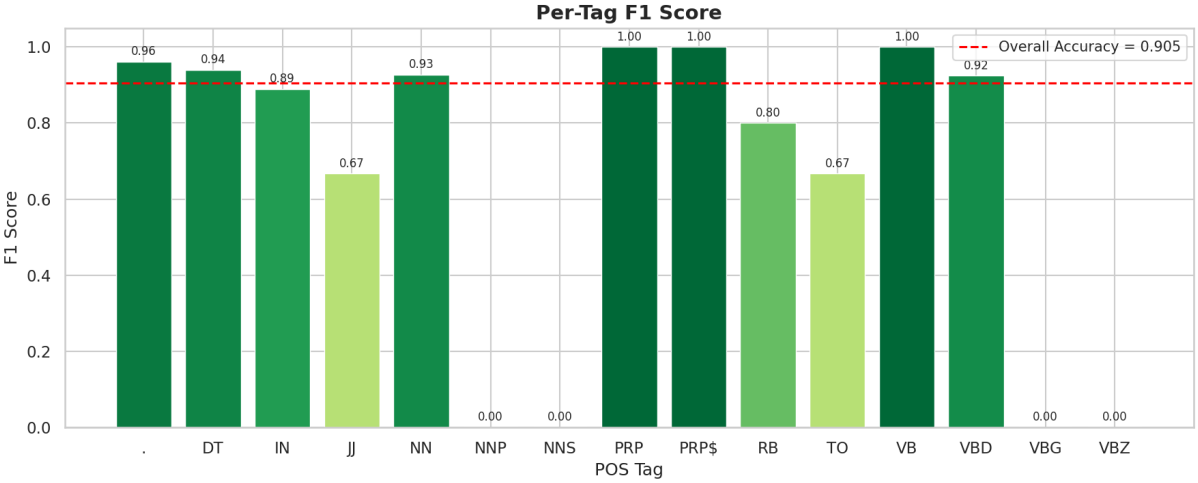

Figure 2: Per-Tag F1 Score. Bars are coloured from red (low) to green (high). The dashed red line marks overall token accuracy.

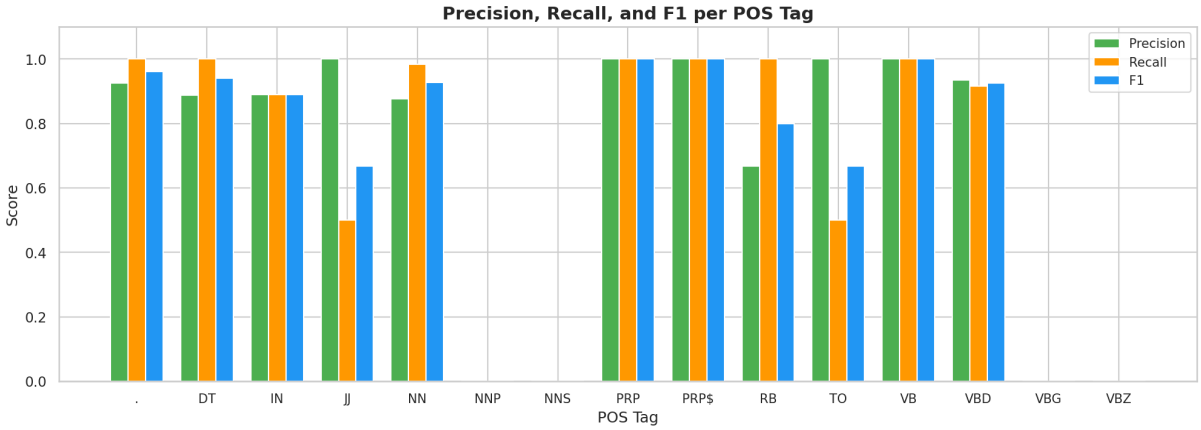

Figure 3: Precision, Recall and F1 per POS Tag. Grouped bars allow a direct comparison of all three metrics.

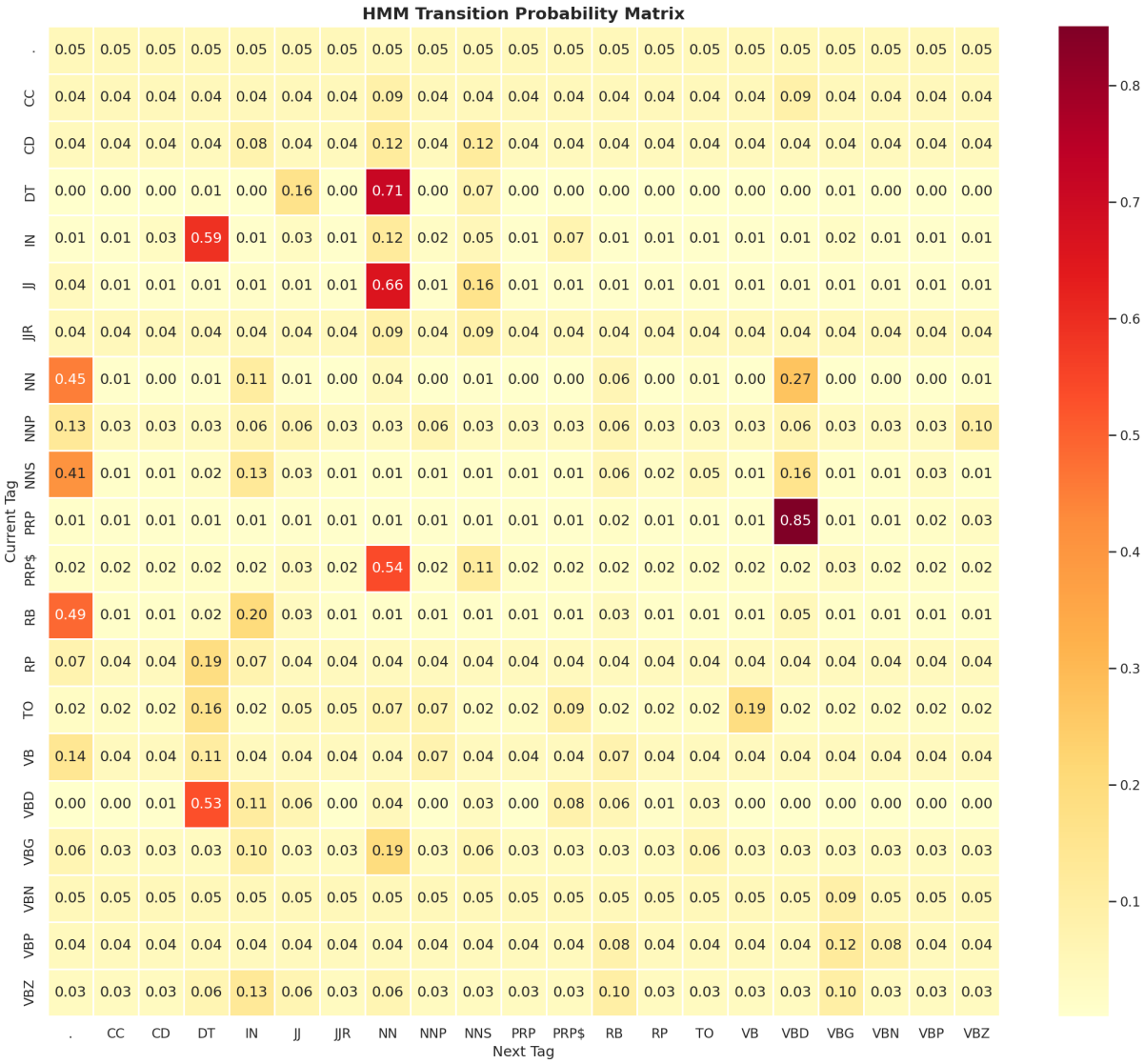

Figure 4: HMM Transition Probability Matrix. Cell (i, j) shows the smoothed probability of transitioning from tag i to tag j.

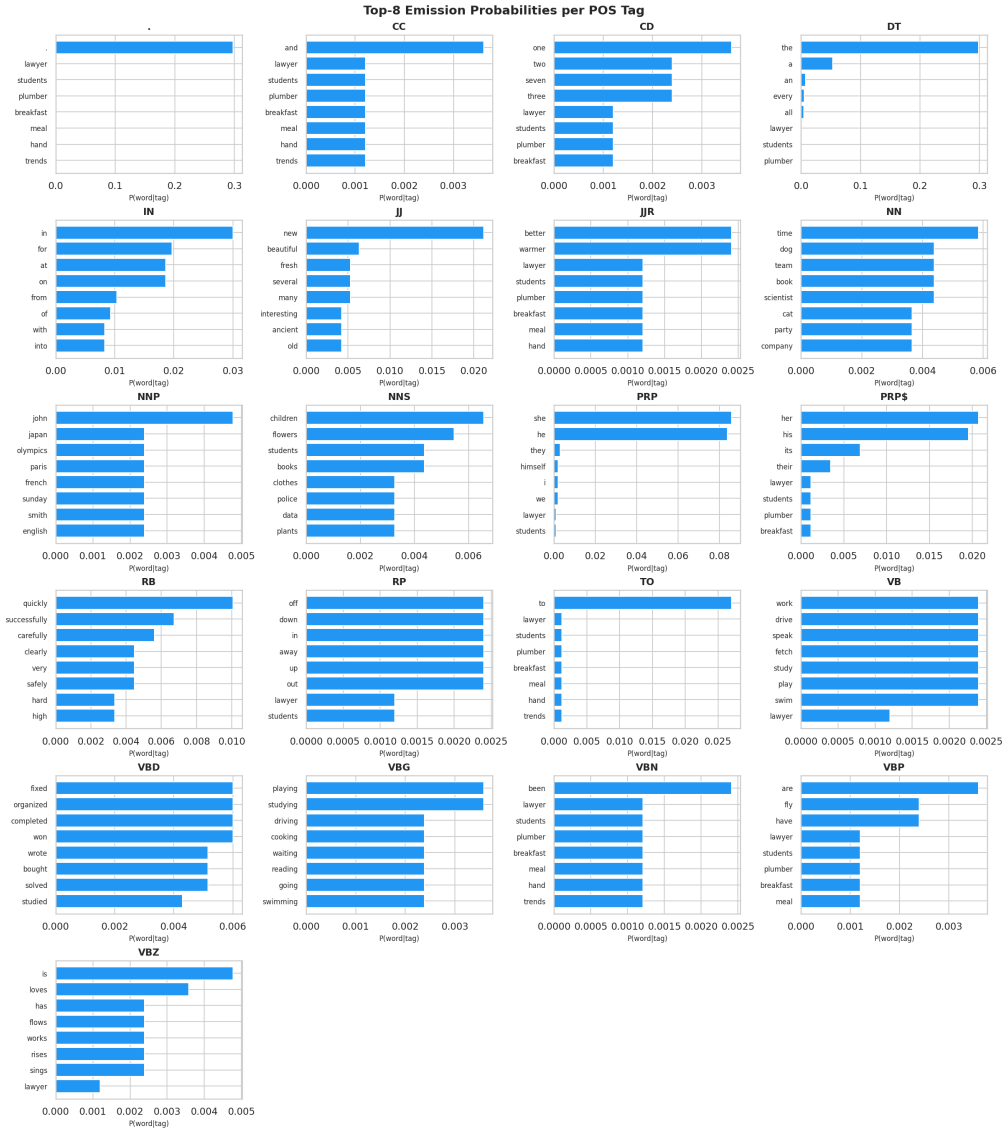

Figure 5: Top Emission Probabilities per Tag. Each sub-plot shows the most likely words for a given POS tag, reflecting the model's learned word-class associations.

7. Discussion

The confusion matrix (Figure 1) reveals where the tagger makes most errors. Common confusions occur between tags with overlapping lexical evidence, for example between *NN* (singular noun) and *NNP* (proper noun), or between *VBD* (past-tense verb) and *VBN* (past-participle). These ambiguities are inherent to POS tagging and can only be resolved with longer-range context that a first-order HMM cannot fully exploit.

The transition matrix (Figure 4) shows linguistically sensible patterns: determiners (DT) are most often followed by adjectives (JJ) or nouns (NN), prepositions (IN) are followed by determiners (DT) or nouns, and verbs (VBD, VBZ) are preceded by pronouns (PRP) or nouns.

The emission probabilities (Figure 5) show that each tag has characteristic vocabulary: *DT* is dominated by 'the', 'a', 'an'; *PRP* by 'he', 'she', 'they'; and *VBD* by common past-tense verbs such as 'ran', 'said', 'went'.

8. Conclusions

This project demonstrated a complete end-to-end pipeline for HMM-based POS tagging using the Viterbi algorithm. Key observations:

- Laplace smoothing is essential for handling unseen word–tag combinations and out-of-vocabulary words.
- The Viterbi decoder reliably finds the globally optimal tag sequence in polynomial time.
- A first-order HMM captures local syntactic regularities effectively but is limited by its Markov assumption to a single preceding tag.
- Higher accuracy can be achieved with higher-order HMMs, discriminative models (e.g., CRF, BiLSTM-CRF), or pre-trained language models (e.g., BERT), at the cost of additional complexity.

Complementary suggestions (items not explicitly requested but recommended for a production system):

- Cross-validation to obtain robust accuracy estimates.
- Unknown-word morphological features (suffixes, capitalisation) to improve OOV handling.
- Beam search as a faster approximate alternative to exact Viterbi for large tag sets.
- A web-based demo interface using Flask or Gradio.
- Integration with a standard benchmark (Penn Treebank WSJ sections 02–21 for training, section 23 for testing).